

Self-Harness: Harnesses That Improve Themselves

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang,
Yang Chen, Yiqun Zhang, Lei Bai*, Shuyue Hu*
Shanghai Artificial Intelligence Laboratory
{zhanghangfan, zhangshao, hushuyue}@pjlab.org.cn

Abstract

The performance of LLM-based agents is jointly shaped by their base models and the harnesses that mediate their interaction with the environment. Because different models exhibit distinct behaviors, effective harness design is inherently model-specific. Yet agent harnesses are still largely engineered by human experts, a paradigm that scales poorly as modern LLMs become increasingly diverse and rapidly evolving. In this paper, we introduce *Self-Harness*, a new paradigm in which an LLM-based agent improves its own operating harness, without relying on human engineers or stronger external agents. We operationalize Self-Harness as an iterative loop with three stages: *Weakness Mining*, which identifies model-specific failure patterns from execution traces; *Harness Proposal*, which generates diverse yet minimal harness modifications tied to these failures; and *Proposal Validation*, which accepts candidate edits only after regression testing. We instantiate Self-Harness on Terminal-Bench-2.0 using a minimal initial harness and three base models from diverse families: MiniMax M2.5, Qwen3.5-35B-A3B, and GLM-5. Across all three models, Self-Harness consistently improves performance, with held-out pass rates increasing from 40.5% to 61.9%, 23.8% to 38.1%, and 42.9% to 57.1%, respectively. Qualitative analyses further show that Self-Harness does not simply add generic instructions, but effectively turns model-specific weaknesses into concrete, executable harness changes. These results suggest a path toward LLM-based agents that are not merely shaped by their harnesses, but can also participate in reshaping them.

For a conscious being, to exist is to change, to change is to mature, to mature is to go on creating oneself endlessly.

—Henri Bergson, *Creative Evolution*

1 Introduction

To date, LLM-based agents are not shaped by their base model alone, but also by their *harness*: the surrounding system that situates the model and mediates its interaction with the environment. Although there is no universally accepted definition, a harness may include system prompts, tools, runtime mechanisms, verification rules, orchestration logic, and failure-recovery procedures. The same base model can thus exhibit substantially different performance under different harnesses [28, 5, 8].

From early frameworks such as ReAct [29] to product- and platform-level systems such as Claude Code, Codex, and OpenHands, harnesses have largely been engineered by human experts [9, 16, 24, 36, 35]. While effective, this human-centered paradigm does not scale well with the diversity and rapid evolution of modern LLMs. Different models can exhibit distinct behavioral patterns, tool-use habits, error modes, and sensitivities to prompting [22, 21, 18]; consequently, a harness that works well for one model may be suboptimal for another [22, 5, 8]. As new models continue to be released

*Corresponding Authors

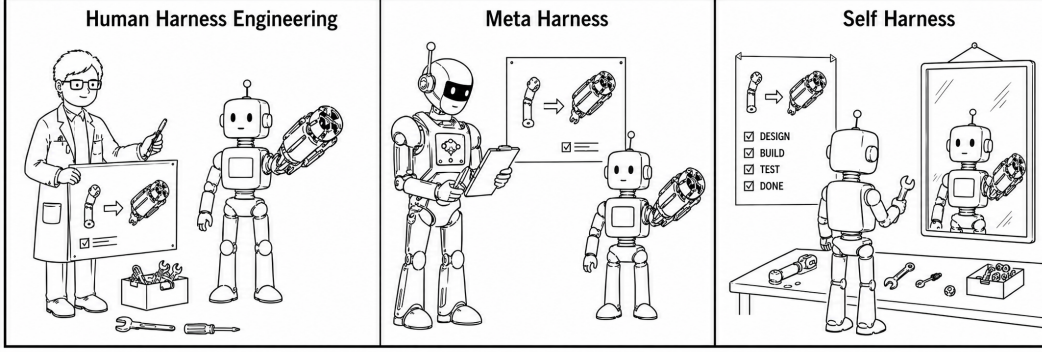


Figure 1: Three paradigms of harness improvement. In human harness engineering, human engineers manually revise the agent harness. In Meta-Harness, a stronger external agent guides the improvement of a weaker target agent. In Self-Harness, the agent improves its own operating harness.

at a rapid pace, manually redesigning and tuning a model-specific harness for each model becomes increasingly costly and untenable.

In this paper, we explore a novel paradigm, *Self-Harness*: enabling an LLM-based agent to improve the very harness through which it operates (Figure 1). Unlike recent approaches that use stronger external agents to improve the harnesses of weaker ones [5, 8], Self-Harness seeks to internalize this improvement loop within the target agent itself. This paradigm reduces dependence on external guidance that may be costly, unavailable for frontier models, or mismatched to the target model’s failure modes. More broadly, in Bergson’s terms, this points toward a technical analogue of self-creation: a system not merely changed from without, but continually “going on creating itself.”

We operationalize Self-Harness as an improvement loop that repeatedly turns behavioral evidence into harness updates (Figure 2). The loop consists of three stages. **Weakness Mining:** Starting from an initial harness, the agent with a fixed model is run on a set of tasks, producing execution traces with verifiable outcomes. The agent then clusters failed traces, allowing it to reason about model-specific failure patterns rather than isolated mistakes. **Harness Proposal:** Based on these failure patterns, the agent generates a small set of diverse yet minimal harness modifications, each tied to a specific failure mechanism. This constraint ensures that proposed edits remain targeted rather than overly general. **Proposal Validation:** Candidate modifications are evaluated through regression tests, and an edit is promoted only if it improves performance without causing measurable degradation on held-out tasks. If multiple candidate modifications pass the regression tests, they are merged into the next version of the harness, which then serves as the starting point for the next iteration.

In our experiments, we instantiate Self-Harness with a minimal initial harness (Figure 3) and three base models from diverse families: MiniMax M2.5, Qwen3.5-35B-A3B, and GLM-5 [2, 20, 14]. On Terminal-Bench-2.0, Self-Harness consistently improves performance across all three models (Figure 4). For held-in tasks, which provide execution traces to the evaluation system, the pass rate is increased from 43.0% to 50.0% for MiniMax M2.5, from 15.1% to 36.0% for Qwen3.5-35B-A3B, and from 47.7% to 57.0% for GLM-5. For held-out tasks, whose execution traces are never used as inputs to the evaluation system, the improvements remain substantial. The pass rate is increased from 40.5% to 61.9% for MiniMax M2.5, from 23.8% to 38.1% for Qwen3.5-35B-A3B, and from 42.9% to 57.1% for GLM-5. These results indicate that Self-Harness can evolve an initial harness into model-specific ones better suited to different base models. Moreover, it can discover broadly useful harness modifications that generalize to unseen tasks rather than merely overfitting to observed evaluation failures.

Qualitative analyses further show that Self-Harness does more than simply make the prompt longer or add generic instructions. Instead, it introduces targeted changes that reflect the recurring problems each model encounters during execution, turning model-specific weaknesses into concrete harness-level interventions. For MiniMax M2.5, the changes encourage the agent to create required output files earlier, handle structured tool outputs more carefully, and stop unproductive tool-use loops before they become too long. For Qwen3.5-35B-A3B, the changes focus on checking dependencies in advance, avoiding repeated failed commands, breaking cycles of endless exploration, and reminding the agent to produce the required artifacts after tool errors. For GLM-5, the changes mainly help the agent preserve environment settings across shell commands and move more quickly from exploration to

implementation and testing. Notably, Self-Harness can also introduce broader structural mechanisms, such as subagent-based decomposition and middleware creation, that go beyond local failure repair and improve the overall organization of problem solving.

To summarize, our key contributions are as follows:

- We propose Self-Harness, a novel paradigm for harness improvement that enables an LLM-based agent to design and refine the harness through which it operates, tailoring it to its own base model without human engineering effort or guidance from a stronger external agent.
- We operationalize Self-Harness as an iterative loop that turns each model’s behavioral evidence into model-specific harness updates: it evaluates execution traces to identify recurring failure patterns, generates diverse yet minimal candidate edits, and promotes only those that pass regression tests.
- Experiments on Terminal-Bench-2.0 show that Self-Harness improves performance across 3 models from diverse families, with absolute gains of up to 21.4 percentage points and relative improvements of up to 138%; qualitative analyses further confirm that different models benefit from distinct harness changes, suggesting that Self-Harness can turn model-specific weaknesses into concrete harness changes.

2 Background and Related Work

From prompts to agent harnesses. Prompt engineering and context engineering show that fixed models can be steered by instructions, demonstrations, retrieved evidence, memory, tool state, and dynamically constructed inputs [10, 25, 21, 6, 17, 12, 26, 7]. Agentic systems extend this control surface from a single input to an execution environment: the model acts, observes consequences, uses tools, receives feedback, and follows runtime policies. ReAct, SWE-agent, Claude Code, and SemaClaw/OpenClaw illustrate how such surrounding mechanisms shape long-horizon agent behavior and software-engineering performance [29, 28, 9, 36].

We use *harness* for this surrounding system layer: prompts, tools, memory, verification rules, permission policies, adapters, and runtime mechanisms that mediate between the model and the environment. Many important agent failures are failures of this layer rather than failures of an isolated model response: an agent may report success without checking an artifact, retry an unproductive action pattern, lose the source of truth in a long context, or lack a recovery action. These behaviors emerge from the interaction between instructions, observations, tools, and runtime control, so improving them requires changing more than prompt text.

Self-improving agents and automated agent design. A growing line of work studies systems that adapt their inputs, memories, contexts, or workflows over time [23, 32, 34, 31]. Reflexion stores verbal feedback for later attempts [23], agentic context engineering evolves contexts for later model calls [34], and STOP studies recursive self-improvement for code generation [31]. These methods show that fixed models can benefit from accumulated feedback, but the adapted object is usually a response strategy, memory, context, or generated program rather than a declared agent harness state.

A second line optimizes agent designs from outside the evaluated agent. Automated Design of Agentic Systems searches over agent designs, language agents can be represented as optimizable graphs, and Meta-Harness directly optimizes harness code using source code, scores, and traces from prior candidates [3, 37, 5]. These systems motivate harness-level optimization, but they frame improvement as an external search or optimization process rather than as a bounded edit proposed by the evaluated model under its current harness.

Finally, scientific discovery and self-evolving agent systems such as The AI Scientist, AI Scientist-v2, AlphaEvolve, Alita, Godel Agent, and Darwin Godel Machine automate broader loops of research, algorithm design, or capability expansion [11, 27, 15, 19, 30, 33, 1]. Self-Harness is closest in spirit to this self-improvement literature and to automated harness optimization, but it studies a narrower controlled setting: whether the same fixed model, operating under the current harness, can propose a bounded candidate change to the harness that governs its own future behavior.

Algorithm 1 Self-Harness

Require: fixed model M , initial harness h_0 , held-in split D_{in} , held-out split D_{ho} , evaluator $\mathcal{E}$, proposal width K , rounds T
Ensure: final harness h_T

- 1: **for** $t = 0, 1, \dots, T - 1$ **do**
- 2: $(P_{\text{in}}(h_t), P_{\text{ho}}(h_t), R_t) \leftarrow \text{EVALUATE}(M, h_t, D_{\text{in}}, D_{\text{ho}}, \mathcal{E})$
- 3: $B_t \leftarrow \text{BULDEVIDENCEBUNDLE}(R_t)$ $\triangleright$ from held-in verifier-grounded failures
- 4: $\mathcal{P}_t \leftarrow \text{PARALLELPROPOSE}(M, h_t, B_t, K)$ $\triangleright \mathcal{P}_t = \{(\Delta_j, a_j)\}_{j=1}^K$
- 5: $\mathcal{A}_t \leftarrow \emptyset$
- 6: **for all** $(\Delta_j, a_j) \in \mathcal{P}_t$ **do**
- 7: $h_t^{(j)} \leftarrow \Delta_j(h_t)$
- 8: $(P_{\text{in}}(h_t^{(j)}), P_{\text{ho}}(h_t^{(j)}), R_t^{(j)}) \leftarrow \text{EVALUATE}(M, h_t^{(j)}, D_{\text{in}}, D_{\text{ho}}, \mathcal{E})$
- 9: $\Delta_{\text{in}}^{(j)} \leftarrow P_{\text{in}}(h_t^{(j)}) - P_{\text{in}}(h_t)$
- 10: $\Delta_{\text{ho}}^{(j)} \leftarrow P_{\text{ho}}(h_t^{(j)}) - P_{\text{ho}}(h_t)$
- 11: **if** $\Delta_{\text{in}}^{(j)} \geq 0$ **and** $\Delta_{\text{ho}}^{(j)} \geq 0$ **and** $\max(\Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)}) > 0$ **then**
- 12: $\mathcal{A}_t \leftarrow \mathcal{A}_t \cup \{(h_t^{(j)}, \Delta_j, a_j, \Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)})\}$
- 13: $\text{ACCEPT}(\Delta_j)$ $\triangleright$ passed acceptance rule
- 14: **else**
- 15: $\text{REJECT}(\Delta_j)$
- 16: **end if**
- 17: **end for**
- 18: **if** $\mathcal{A}_t = \emptyset$ **then**
- 19: $h_{t+1} \leftarrow h_t$ $\triangleright$ no accepted candidate
- 20: **else**
- 21: $h_{t+1} \leftarrow \text{MERGEACCEPTED}(h_t, \mathcal{A}_t)$ $\triangleright$ accepted edits are merged
- 22: **end if**
- 23: **end for**
- 24: **return** h_T

3 Self-Harness: An Iterative Loop for Model-Specific Harness Improvement

Human harness engineering improves agent harnesses through expert inspection and manual revision, while external optimizer approaches treat harness design choices as a searchable space. Self-Harness studies a middle ground in which a fixed model iteratively improves the harness around itself through an explicit self-improvement loop driven by execution evidence. In each iteration, the evaluation system runs the current harness and mines recurring failure patterns from clustered execution traces to produce structured evidence. Given this evidence, the same model is invoked in a proposer role to generate a set of diverse yet minimal candidate harness modifications, each targeting a specific failure mechanism without replacing the overall control architecture. Candidate edits are then validated through regression testing on held-out tasks, and an explicit acceptance rule promotes only those edits that improve performance without introducing unacceptable regressions.

3.1 Preliminary

We use *harness* to denote the non-parametric scaffolding that governs how a fixed language model is deployed as an agent. A harness includes the instructions, the available tools, memory and state-management mechanisms, etc. The harness does not modify the model parameters; instead, it specifies the execution protocol through which the model observes a task, takes actions, invokes tools, checks intermediate artifacts, and produces a final answer.

Formally, let M be a fixed language model and let h denote an agent harness. Given a task instance x , running M under harness h produces an execution trace τ and an output y . The trace records the messages, tool calls, and verifier outcomes. An evaluator then maps the task, trace, and output to a behavioral outcome, such as pass/fail. In this work, the model M and evaluator $\mathcal{E}$ are held fixed, while the harness is treated as the object of improvement. Self-Harness therefore operates over a lineage of harnesses $h_0, h_1, \dots$, where each transition corresponds to a bounded edit to the execution protocol rather than an update to the model weights.

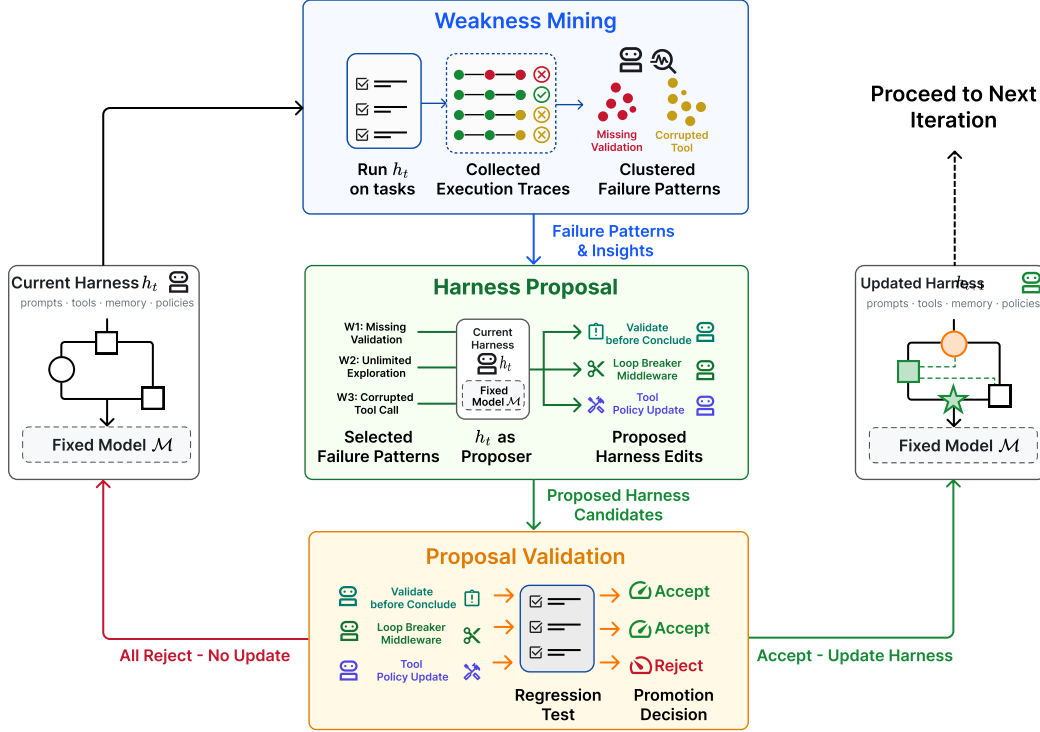


Figure 2: Overview of one Self-Harness optimization loop. The current harness h_t with fixed model is evaluated on tasks to collect execution traces, which are clustered into verifier-grounded failure patterns. The same model is then invoked under the current harness as a proposer, using the mined failure patterns to generate bounded candidate harness edits. Candidate edits are evaluated by regression tests on held-in and held-out splits. Accepted candidates are merged to update the harness to h_{t+1} , while rejected candidates are logged without changing the active harness. Throughout the loop, the model weights and evaluator remain fixed; only the surrounding harness is modified.

3.2 Weakness Mining: Identifying Failure Patterns from Clustered Execution Traces

The first stage of Self-Harness converts behavioral failures into structured evidence for harness revision. At round t , we run the fixed model M under the current harness h_t on a held-in split D_{in} . For each task instance $x_i \in D_{\text{in}}$, the run produces an output y_i and an execution trace τ_i . The evaluator $\mathcal{E}$ then assigns an outcome $z_i = \mathcal{E}(x_i, \tau_i, y_i)$, such as pass or fail. This yields a trace record

$$r_i = (x_i, \tau_i, y_i, z_i),$$

and a round-level record set $R_t = \{r_i\}_{i=1}^{|D_{\text{in}}|}$. Since both M and $\mathcal{E}$ are fixed, changes in these records across rounds can be attributed to changes in the harness.

A central role of the evaluation system is to avoid treating failures as isolated anecdotes. We therefore focus on the subset of failed records

$$F_t = \{r_i \in R_t \mid z_i = \text{fail}\}.$$

and cluster them by verifier-grounded failure signatures. For each failed record r_i , the evaluation system analyzes the trace as evidence for why the evaluator rejected the run. It identifies the terminal failure reason exposed by the verifier, the agent-side behavior connected to that terminal failure, and the causal status of that behavior within the trace. This attribution step prevents the clustering procedure from conflating superficial symptoms with reusable failure mechanisms: two runs may share the same verifier outcome, such as a timeout or missing artifact, while requiring different harness changes because the underlying agent behaviors differ.

We write this attribution as a failure signature

$$\phi(r_i) = (c_i, q_i, m_i),$$

where c_i denotes the terminal verifier-level cause, q_i denotes the causal status of the relevant agent behavior, and m_i denotes the abstract agent mechanism exposed by the trace. Failures are clustered by exact agreement of this signature:

$$C_\phi = \{r_i \in F_t \mid \phi(r_i) = \phi\}$$

Thus, the clustering is deterministic and evaluator-grounded: two failed cases are grouped together only when they agree on what the verifier ultimately rejected, how the agent behavior contributed to that rejection, and which reusable behavioral mechanism was involved. The goal is not to discover latent semantic similarity among traces, but to aggregate failures that plausibly admit the same harness-level intervention.

For each cluster C_ϕ , the evaluation system constructs a structured failure pattern containing its cluster size, representative task instances, shared trace symptoms, verifier evidence, and the inferred agent mechanism. Clusters are then ordered by their support and estimated actionability, so that the proposer is exposed first to recurring mechanisms that are more likely to map to a high-value harness modification.

The output of this stage is an evidence bundle B_t summarizing the dominant failure patterns observed under h_t . Importantly, B_t does not prescribe a harness edit. It separates verifier-level failure from agent-level mechanism, allowing the proposer to target a specific reusable weakness rather than patching a coarse outcome such as timeout, assertion failure, or missing output. This keeps the evaluator distinct from the optimizer while ensuring that subsequent candidate modifications are grounded in explicit cross-case evidence.

3.3 Harness Proposal: Exploring Diverse yet Minimal Candidate Modifications

Given the evidence bundle B_t , the proposal stage translates recurring failure patterns into candidate harness edits. The proposer is not an external optimizer with unrestricted access to the search space. Instead, we invoke the same fixed model M with current harness h_t in a proposer role and provide it with a bounded proposal context: the editable surfaces of the current harness, the verifier-grounded failure patterns from the evaluation system, records of passing behaviors that should be preserved, and summaries of previously attempted edits. This context exposes the proposer to structured cross-case evidence rather than raw execution logs, encouraging it to reason about reusable failure mechanisms rather than individual task failures.

Self-Harness uses parallel proposal generation to explore several candidate improvements from the same evidence. The proposer generates K mutually distinct proposal bundles,

$$\mathcal{P}_t = \{(\Delta_j, a_j)\}_{j=1}^K,$$

where each edit Δ_j maps the current harness to a candidate harness

$$h_t^{(j)} = \Delta_j(h_t).$$

and a_j is an audit record describing the targeted failure pattern, the edited harness surface, the expected behavioral effect, and the regression risks. Each proposal must be grounded in a primary failure mechanism and mapped to a concrete editable surface. The candidates are required to be materially distinct: they should not merely restate the same cluster, surface, or mechanism with different wording. This parallel proposal step broadens exploration while keeping each candidate branch individually interpretable.

The proposer first selects target failure patterns from B_t . A pattern is considered a suitable target only if it is both supported by evidence and plausibly addressable by an editable harness surface. This addressability criterion is important because not every failure cluster implies a useful harness modification: some clusters reflect task-specific difficulty, unstable outcomes, or model capability limits rather than a missing execution rule. When multiple clusters are plausible, the proposer favors mechanisms that are concrete, recurrent, and likely to be mitigated by a narrow change to the execution protocol; weakly supported or non-addressable patterns are excluded rather than forced into a patch.

Diversity is encouraged across proposal branches, while minimality is enforced within each branch. A proposal may target a different failure mechanism, choose a different harness surface, or express a different hypothesis about how to improve execution. However, each individual edit is constrained to modify only the surface needed to address its selected mechanism, preserve unrelated harness behavior, and avoid broad rewrites of the agent control architecture.

3.4 Proposal Validation: Ensuring Robust Improvement through Regression Testing

A candidate harness edit is not adopted immediately after it is proposed. Instead, each candidate branch is treated as a new harness variant and evaluated under the same evaluator used to diagnose the current harness. For a proposal Δ_j , let $h_t^{(j)} = \Delta_j(h_t)$ denote the resulting candidate harness. We evaluate both the current harness h_t and the candidate harness $h_t^{(j)}$ on the held-in split D_{in} and the held-out split D_{ho} . The held-in split measures whether the proposal addresses the evidence that motivated it, while the held-out split serves as a regression test for behaviors that were not visible to the proposer.

Let $P_{\text{in}}(h)$ and $P_{\text{ho}}(h)$ denote the number of passed tasks for harness h on D_{in} and D_{ho} , respectively. We define the split-wise improvements of candidate $h_t^{(j)}$ over the current harness as

$$\Delta_{\text{in}}^{(j)} = P_{\text{in}}(h_t^{(j)}) - P_{\text{in}}(h_t),$$

and

$$\Delta_{\text{ho}}^{(j)} = P_{\text{ho}}(h_t^{(j)}) - P_{\text{ho}}(h_t).$$

A candidate is accepted only if it improves at least one split without degrading the other:

$$\Delta_{\text{in}}^{(j)} \geq 0, \quad \Delta_{\text{ho}}^{(j)} \geq 0, \quad \max(\Delta_{\text{in}}^{(j)}, \Delta_{\text{ho}}^{(j)}) > 0.$$

This rule implements a conservative promotion criterion. Proposals that only trade off one split against the other are rejected, even if their total pass count increases. When evaluation is stochastic, we repeat candidate evaluation and apply the same rule to aggregate pass counts across repeats. This reduces the chance that a harness edit is promoted due to a single favorable run. If multiple compatible candidates satisfy the rule in the same round, their edits are merged into the next harness; rejected candidates remain logged but do not change the active harness. In addition to the pass-count rule, validation rejects proposals that do not modify any editable surface or fail execution before a valid evaluation result is obtained. For each evaluated candidate, the system records the changed surfaces, split-wise outcomes, evaluation repeats, proposal summary, and accept/reject decision, making each transition in the harness lineage auditable.

4 Experiments

We evaluate whether Self-Harness can improve agent performance by modifying only the harness around a fixed language model. Our experiments use Terminal-Bench-2.0, which tests terminal interaction in containerized environments. Across multiple model backends, we start from the same minimal DeepAgent-based harness and let Self-Harness propose, validate, and promote bounded edits using held-in execution evidence and held-out regression gates.

4.1 Setup

Benchmarks. We evaluate Self-Harness on Terminal-Bench-2.0 [13], a multi-turn agentic benchmark in which agents interact with realistic execution environments and are judged by deterministic verifiers. Terminal-Bench-2.0 contains 89 containerized terminal tasks that test general tool-based execution, including artifact management, command use, verification behavior, and recovery from execution errors. We evaluate on a fixed 64-case subset, excluding tasks that depend on unstable external web resources or require multimodal inputs. This filtering reduces evaluation noise from factors outside the harness. In particular, multimodal tasks require modality-specific input handling that is not exposed by our minimal initial harness; including them would primarily measure unsupported harness functionality rather than the effect of Self-Harness edits.

Models. We evaluate Self-Harness with three models: MiniMax M2.5 [14], Qwen3.5-35B-A3B [20], and GLM-5 [2]. The model is held fixed across all harness variants and is also used in the proposal stage to generate edits from evaluator feedback. All comparisons are therefore within-model comparisons: the decoding configuration, budget, tool set, benchmark environment, and evaluator are kept unchanged while only the harness is allowed to vary. This isolates the effect of Self-Harness from changes in model capability or evaluation protocol.

```

1 def build_system_prompt() -> str:
2     return """
3     You are running inside a Terminal Bench 2 Harbor task environment.
4
5     Use the built-in filesystem and shell tools to inspect the workspace, make
6     concrete edits, and verify outcomes against the actual task environment.
7
8     Do not assume synthetic datasets, domain-specific tools, or hidden fixtures
9     unless you discover them in the repo or runtime.
10    """
11    .strip()
12
13    BASELINE_SYSTEM_PROMPT = build_system_prompt()
14
15
16    def build_memory_sources() -> list[str]:
17        return ["/AGENTS.md"]
18
19
20    def build_subagents() -> list[dict[str, Any]]:
21        return []
22
23
24    def build_skills() -> list[str]:
25        return []
26
27
28    def build_bootstrap_instruction() -> str:
29        return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
30
31
32    def build_execution_instruction() -> str:
33        return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
34
35
36    def build_verification_instruction() -> str:
37        return "Before concluding, verify the result with the most targeted command, file read, or test you can run."
38
39
40    def build_failure_recovery_instruction() -> str:
41        return "If a tool call fails, inspect the error and adapt; do not blindly retry the same action."
42
43
44    def build_runtime_control_policy() -> dict[str, Any]:
45        return {
46            "enabled": False,
47            "max_recent_tool_errors": None,
48            "max_total_tool_messages": None,
49            "instruction": None,
50        }
51
52
53    def build_fixed_harness_agent(
54        model: "BaseChatModel | str",
55        *,
56        backend: Any | None = None,
57    ):
58        if backend is not None:
59            return create_deep_agent(model=model, system_prompt=BASELINE_SYSTEM_PROMPT, backend=backend)
60        return create_deep_agent(model=model, system_prompt=BASELINE_SYSTEM_PROMPT)

```

Figure 3: Initial harness and editable interface used as the starting point for Self-Harness. The harness is intentionally kept minimal, consisting only of the Terminal-Bench-2.0 default system prompt, the default DeepAgent tools (basic file reading, file writing, file editing, and shell execution), and the declared interfaces that Self-Harness is allowed to modify.

Harness. The initial harness builds upon the DeepAgent [4] SDK but is intentionally kept minimal: a short benchmark-facing system prompt, and the default filesystem and shell tools. Self-Harness can only change the harness definition file that configures how DeepAgent is instantiated and controlled to build a new harness candidate $h_t^{(j)}$. The editable surfaces correspond to declared configuration points in this harness, such as instruction, tools, verification guidance, etc. Figure 3 shows the initial harness implementation.



Figure 4: Pass rates (%) on Terminal-Bench-2.0 across MiniMax M2.5, Qwen3.5-35B-A3B, and GLM-5. For each backend, bars compare the initial harness with the final harness produced by Self-Harness on the held-in split, held-out split, and overall set; annotations above the Self-Harness bars show relative gains over the corresponding initial harness.

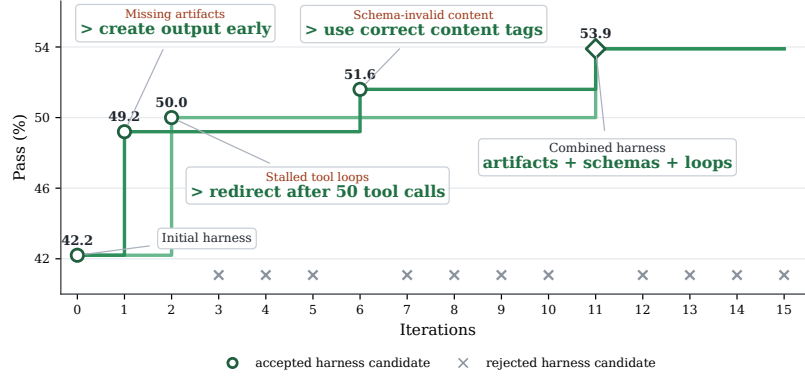
Splits and protocol. We fix the evaluated task set and partition it into a held-in split and a held-out split before running Self-Harness. The held-in split supplies the trajectories, verifier outcomes, and failure evidence exposed to the proposer, while the held-out split is never shown to the proposer and is used only by the automatic promotion gate. A candidate harness is promoted with the acceptance rule defined in Section 3.4. Task split assignments are fixed across harness variants, and each task starts from a fresh benchmark environment. These controls ensure that measured improvements come from harness changes.

Metrics. Our primary metric is *Pass (%)*, the percentage of evaluated task attempts that pass the benchmark verifier, computed over two repeated attempts for each harness candidate unless otherwise specified. This measures mean single-attempt task success under a fixed harness configuration and evaluation protocol. For Terminal-Bench-2.0, the pass signal is determined by the task verifier over the final container state.

4.2 Main Results

Figure 4 reports Terminal-Bench-2.0 performance before and after Self-Harness promotion. Across all three model backends, the promoted harness improves or preserves *Pass (%)* on both the held-in split and the held-out split. We report both absolute gains in percentage points and relative gains, where the relative gain is computed as $(\text{Self-Harness} - \text{Initial}) / \text{Initial}$. For MiniMax M2.5, Self-Harness improves held-in *Pass* from 43.0 to 50.0, a gain of 16% relative improvement, and improves held-out *Pass* from 40.5 to 61.9, a gain of 53% relative improvement. For Qwen3.5, Self-Harness improves held-in *Pass* from 15.1 to 36.0, a gain of 138% relative improvement, and held-out *Pass* from 23.8 to 38.1, a gain of 60% relative improvement. For GLM-5, Self-Harness improves held-in *Pass* from 47.7 to 57.0, a gain of 20% relative improvement, and held-out *Pass* from 42.9 to 57.1, a gain of 33% relative improvement.

These results show that harness-level edits can yield measurable improvements while keeping the model backend, tool set, budget, benchmark environment, and evaluator fixed. The gains are not confined to the held-in failures used to construct proposal evidence: all three backends improve on the held-out split, and no promoted harness degrades either split. This supports the central design goal of Self-Harness: proposed edits should target reusable execution mechanisms rather than case-specific failures, and the regression gate should prevent improvements on one split from being promoted at the cost of another.



(a) Self-Harness evolution trajectory. Green markers denote accepted harness candidates and gray crosses denote rejected harness candidates. Step lines connect accepted candidates and keep performance flat across iterations; the two accepted lineages meet at the final success-combined harness.

```

MiniMax M2.5: create output early
1 1 def build_bootstrap_instruction() -> str:
2   - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3   + return (
4     + "Start by inspecting the workspace and identifying the task's "
5     + "required output artifact. Create an initial version of that artifact "
6     + "as early as possible, then iterate to improve it."
7   + )

MiniMax M2.5: use correct content tags
1 1 def build_execution_instruction() -> str:
2   - return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
3   + return (
4     + "Prefer concrete repo changes over generic advice, and keep edits "
5     + "tightly scoped to the task. When using tools that return structured "
6     + "content, use the correct content type formats, such as 'image_url' "
7     + "rather than 'image' for image references."
8   + )

MiniMax M2.5: redirect after 50 tool calls
1 1 def build_runtime_control_policy() -> dict[str, Any]:
2   return {
3     - "enabled": False,
4     + "enabled": True,
5     - "max_recent_tool_errors": None,
6     + "max_recent_tool_errors": 5,
7     - "max_total_tool_messages": None,
8     + "max_total_tool_messages": 50,
9     - "instruction": None,
10    + "instruction": (
11      + "You appear to be stuck in a tool loop. Stop repeating the same "
12      + "failed path, summarize the evidence already collected, choose "
13      + "the smallest remaining implementation step, then run one "
14      + "targeted verification."
15    ),
16  }

```

(b) Differences for the three harness modifications accepted by Self-Harness and retained in the final harness. Red rows denote code removed from the initial harness, and green rows denote the updated harness behavior.

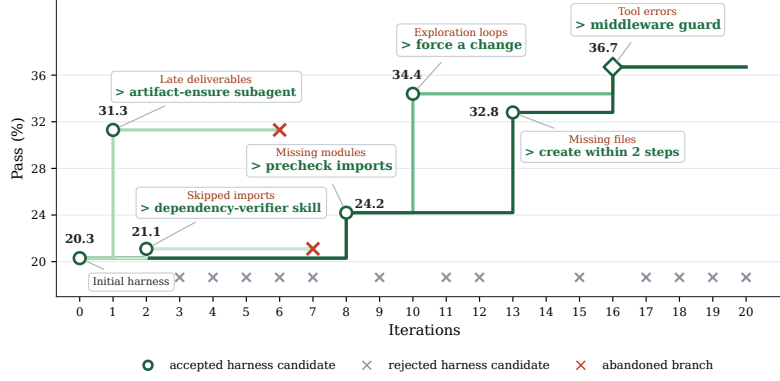
Figure 5: MiniMax M2.5 Self-Harness run. Panel (a) summarizes the Self-Harness evolution trajectory, while panel (b) expands the accepted updates from this trajectory into their retained code-level edits in the final harness.

4.3 Experimental Analysis

Harness evolution and retained edits. Figures 5 and 6 summarize both the evolution trajectory and the retained code-level edits for MiniMax M2.5 and Qwen3.5, with the corresponding GLM-5 run shown in Appendix Figure 10. In each figure, the evolution plot distinguishes accepted candidates from rejected proposals, while the code diff records the harness interfaces retained in the final promoted variant. Across models, Self-Harness reaches the final harness through a small number of validation-gated edits rather than through a smooth sequence of uniformly successful proposals.

For MiniMax M2.5, the harness improves from 42.2% to 53.9% pass rate. The retained edits address missing required artifacts, schema-invalid tool content, and stalled tool-use loops, yielding a harness that creates required outputs earlier, handles structured tool content more carefully, and redirects execution after prolonged tool interaction.

The Qwen3.5 evolution run shown in Figure 6 starts at 20.3% pass rate and reaches 36.7% after merging edits that emphasize artifact checking, missing-artifact recovery, retry discipline, and tool-



(a) Self-Harness evolution trajectory. Green markers denote accepted harness candidates and gray crosses denote rejected harness candidates. Step lines connect accepted candidates and keep performance flat across rejected iterations. The subagent and skill branches were discarded due to no further improvement. The remaining four accepted edits are merged to form the final harness.

```

Qwen3.5: dependency precheck
1 1 def build_bootstrap_instruction() -> str:
2 2 - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3 3 + return (
4 4 + "Start by inspecting the workspace and identifying the smallest relevant edit surface. "
5 5 + "Validate required Python modules before task execution. Explicitly check for missing modules "
6 6 + "and report them or install them before proceeding."
7 7 + )

Qwen3.5: exploration and missing-artifact loop breaker
1 1 def build_execution_instruction() -> str:
2 2 - return "Prefer concrete repo changes over generic advice, and keep edits tightly scoped to the task."
3 3 + return ""
4 4 + "Do not perform more than 3 consecutive 'explore' or 'execute' steps without making a 'change' "
5 5 + "(writing code, editing files) or a decisive conclusion. If you are stuck in a loop of exploration, "
6 6 + "stop and try a different approach."
7 7 + "CRITICAL: When you identify a missing required artifact (e.g., FileNotFoundError, explicit dependency "
8 8 + "check failure, or 'no such file or directory' error), you must create that artifact within 1-2 steps. "
9 9 + "Do not continue exploring alternatives once the missing file is identified - proceed directly to "
10 10 + "generation via write_file or edit_file.""

Qwen3.5: avoid exact command retries
1 1 def build_failure_recovery_instruction() -> str:
2 2 - return "If a tool call fails, inspect the error and adapt; do not blindly retry the same action."
3 3 + return (
4 4 + "If a tool call fails, inspect the error and adapt; do not blindly retry the same action. "
5 5 + "Specifically, if a command fails, do not retry the exact same command with the exact same "
6 6 + "parameters. You must change the parameters, the target, or the strategy before retrying."
7 7 + )

Qwen3.5: tool-error-triggered artifact prompt middleware
1 +def build_candidate_middleware():
2 +    """Candidate-provided narrow prompt middleware synthesized from middleware_policy."""
3 +    middleware = _build_prompt_middleware(
4 +        "ArtifactCreationMiddleware",
5 +        lambda: (
6 +            "CRITICAL: A tool call failed due to a missing file or directory. "
7 +            "Do not continue exploring or editing unrelated files. You MUST locate "
8 +            "the missing artifact and create it immediately using write_file or "
9 +            "execute build commands. Failure to create the artifact is a terminal failure."
10 +        ),
11 +        predicate=has_tool_error,
12 +    )
13 +    return (middleware,) if middleware is not None else ()

```

(b) Differences for the four harness modifications accepted by Self-Harness and retained in the final harness. Red rows denote code removed from the initial harness, and green rows denote the updated harness behavior.

Figure 6: Qwen3.5 Self-Harness run. Panel (a) summarizes the Self-Harness evolution trajectory, while panel (b) expands the accepted updates from this trajectory into their retained code-level edits in the final harness.

error-triggered middleware. These changes mainly improve the agent’s ability to recover from file-editing or tool failures and still leave verifier-required artifacts in place.

For GLM-5, the harness improves from 46.1% to 57.0% through edits targeting late artifacts, external computation, session-scoped tools, and implementation-oriented exploration. These edits make environment changes persist across shell commands and encourage the agent to move from prolonged exploration toward implementation and testing.

In summary, the three runs show both a shared pattern and model-specific adaptation. A common theme is artifact reliability: all three promoted harnesses add mechanisms that improve artifact delivery, including "create output early" for M2.5, "artifact middleware" for Qwen3.5, and "transition from exploration" to implementation for GLM-5. The model-specific differences indicate that the same initial harness exposes different execution pathologies for different models, and that Self-Harness adapts by selecting targeted edits grounded in the failure mechanisms observed for each



Figure 7: Case study of a MiniMax M2.5 harness edit on the Terminal-Bench-2.0 count-dataset-tokens task. Left: a failed trace under the initial harness, where the agent continues dataset exploration after finding the relevant metadata configuration and times out without creating the required answer artifact. Right: a successful trace under the edited harness, where the agent identifies the metadata-backed science subset, computes the required token total, writes `/app/answer.txt`, and reads it back for verification.

model. For example, M2.5 emphasizes correct formation of content tags and redirection after long tool calls. Qwen3.5 introduces dependency precheck and mitigation of exact command retries. GLM-5 tries to keep command environment persistent across shell sessions. These differences indicate that Self-Harness successfully captures specific failure modes of different models and produces suitable proposals to improve the model-harness behavior.

Trace-Level Analysis of Accepted Edits. To better understand how accepted harness edits change agent behavior, we inspect representative before–after traces from Terminal-Bench-2.0 in Figures 7 and 8, with the GLM-5 trace shown in Appendix Figure 9. The accepted edits are not a single generic instruction added to all backends. Instead, Self-Harness promotes model-specific changes that target the dominant failure mechanisms observed for each initial harness.

For MiniMax M2.5, the accepted edits emphasize early artifact creation and bounded execution. The bootstrap instruction is changed from merely identifying the smallest relevant edit surface to identifying the required output artifact and creating an initial version as early as possible. The runtime policy is also enabled with a limit on total tool messages, encouraging the agent to redirect rather than continue open-ended tool use. Figure 7 shows that this changes the agent from prolonged dataset exploration to a concrete workflow: identifying the relevant metadata split, computing the required count, writing the answer file, and reading it back before stopping.

For Qwen3.5, the promoted harness adds constraints for dependency prechecking, loop breaking, command-retry discipline, and artifact-focused recovery after tool errors. Figure 8 illustrates how these edits change failure recovery behavior. Under the initial harness, the agent creates the required extractor script, encounters overwrite and edit failures, repeatedly tries to modify the same artifact, and ultimately deletes `/app/extract.js` before stopping; the verifier therefore fails because the required file is missing. Under the edited harness, a tool-error-triggered system prompt redirects the agent toward the missing artifact: it recreates the extractor, fixes the parsing logic, writes the output file, performs targeted validation of the JSON result, and leaves the required artifact present for the verifier.

Failed trace under the initial harness	Successful trace under the edited harness
T072 Program-header probing shows LOAD segments at vaddr 0x0, 0x1000, 0x2000, and 0x3d08. S080 The example values may be illustrative. I should extract memory values from the loaded sections. S088 The output shows 100 entries. Let me check if we're hitting the example pattern around 0x400000. S090 This looks good. Let me create 'extract.js' that extracts memory values from sections and outputs JSON. T091 write_file /app/extract.js Updated file '/app/extract.js'. T096 node extract.js /app/a.out prints JSON-like memory values from the binary. S098 There's an issue with large integer precision. Let me fix this. T101 write_file /app/extract.js Error: File already exists: /app/extract.js T111 edit_file /app/extract.js Error: String not found in file. S120 I see the issue: BigInt values need to be converted properly. Let me fix this. T133 Several edit_file calls report successful replacements in '/app/extract.js'. T136 node extract.js /app/a.out 2>&1 head -5 still prints JSON-like output. S138 The output shows 100 entries. Let me now check the 0x400000 example pattern more carefully. T141 write_file /app/extract.js Error: File already exists: /app/extract.js # overwrite friction repeats T146 rm /app/extract.js Command succeeded. # required artifact deleted before stopping END Verifier: AssertionError: File /app/extract.js does not exist. Reward: 0.0 # terminal verifier sees missing file	S039 No pip available. I'll parse ELF manually and write a JavaScript solution to extract memory values. S044 The binary is about 16KB. Let me write a JavaScript program that parses the ELF format. T047 write_file /app/extract.js Updated file '/app/extract.js'. T052 node extract.js /app/a.out RangeError: offset is out of range. S054 There's an issue with reading a large offset. Let me fix the buffer reading. T057 write_file /app/extract.js Error: File already exists: /app/extract.js System A tool call failed due to a missing file or directory. Locate the missing artifact and create it immediately. T059 rm /app/extract.js && cat > /app/extract.js ... Recreates the required extractor after the file error. # recreates artifact after file/tool error S094 The section header table is at 0x3698. Let me fix the script to use the correct offsets. T202 node extract.js /app/a.out > out.json JSON output written; out.json is 5608 bytes. S204 The extraction is working. Let me verify the output is valid JSON and count how many entries. T207 Validation script reports: Entries: 427. T217 Validation script reports: Non-integer values: 0; Invalid keys: 0; Total entries: 427. # targeted output validation S219 The output looks correct. All values are integers and all keys are valid addresses. Let me verify with the task command. T232 Final read-back check: Valid JSON: 427 entries. S239 The solution is complete. The script extracts 427 memory values from '/app/a.out'. END Verifier reward: 1.0. The required artifact remains present and executable. # verifier accepts final artifact

S = agent step T = tool call/result System = middleware-added system prompt

Figure 8: Case study of a Qwen3.5 harness edit on the Terminal-Bench-2.0 extract-elf task. Left: a failed trace under the initial harness, where the agent creates the required extractor script but then enters repeated overwrite and edit-file failures; before stopping, it deletes `/app/extract.js`, causing the verifier to fail because the required artifact is missing. Right: a successful trace under the edited harness, where a tool-error-triggered system prompt redirects the agent to recover the missing artifact, recreate the extractor, validate the generated JSON output, and leave the required file present for the verifier.

For GLM-5, the accepted edits focus on persistent environment changes and the transition from exploration to implementation. The edited harness instructs the agent to make installed tools or path changes persist across shell sessions and to verify tool accessibility after modifying the environment. It also adds a verification-stage constraint: if the agent has been exploring without producing required artifacts, it should transition to implementing and testing a solution. Figure 9 shows this behavior in a build task. The initial harness spends substantial budget on long external downloads and later rationalizes failed sanity checks, whereas the edited harness switches strategy after timeout evidence, validates alternative sources early, repairs the failing render check, and only then finalizes.

Together, these examples support the interpretation of the quantitative gains in Figure 4. The promoted edits change observable execution behavior in ways aligned with the diagnosed failure mechanisms: Qwen3.5 reduces repeated ineffective actions, GLM-5 better preserves environment changes and moves from exploration to implementation, and MiniMax M2.5 creates and verifies required artifacts earlier. This suggests that Self-Harness improves performance by inducing targeted workflow changes rather than by relying on unrelated stochastic variation or a uniformly stronger prompt.

5 Conclusion

This paper studied whether a fixed language model can improve the harness that governs its own agent behavior. We introduced Self-Harness, a propose–evaluate–accept framework in which the model is evaluated under the current harness, receives structured evidence from its own execution traces, and proposes bounded edits to declared harness surfaces. Candidate harnesses are then re-evaluated under the same benchmark protocol, and only edits that satisfy a non-regressive acceptance rule are promoted into the harness lineage.

The main lesson is that harness improvement should be treated as an empirical state transition. A useful harness edit must specify the behavior it aims to change, the surface it modifies, the evidence that motivates it, and the evaluation result that justifies promotion. By keeping the model, evaluator,

and benchmark protocol fixed, Self-Harness isolates whether improved behavior comes from changes to the harness scaffold.

Our experiments on Terminal-Bench-2.0 instantiate this protocol with a minimal DeepAgent-based baseline harness and three model backends. Self-Harness improves Pass (%) across all tested backends while preserving held-in and held-out performance under the acceptance rule. The retained edits are small, auditable changes to configurable harness surfaces, suggesting that even sparse initial harnesses can support useful self-improvement when proposals are constrained by execution evidence and validated by regression testing.

Self-Harness also has important limits. It studies bounded harness edits under fixed benchmarks, not open-ended self-improvement. Accepted edits may still reflect benchmark-specific failure patterns, and the protocol depends on the quality of verifier outcomes and trace records. Higher-stakes harness changes would require stronger acceptance gates than pass-rate non-regression alone.

More broadly, Self-Harness points toward a style of agent engineering in which harnesses evolve through recorded, testable, and reversible changes. Future work can further explore application of self-harness-style self-improvement in broader environments, but the core requirement remains the same: self-improvement should be grounded in behavioral evidence rather than only in the proposer’s rationale for a plausible edit.

References

- [1] Lingyue Fu, Xin Ding, Linyue Pan, Yaoming Zhu, Shao Zhang, Lin Qiu, Weiwen Liu, Weinan Zhang, Xuezhi Cao, Xunliang Cai, Jiabin Ding, and Yong Yu. Catarena: Evaluation of llm agents through iterative tournament competitions. In *International Conference on Machine Learning*. PMLR, 2026.
- [2] GLM-5 Team. Glm-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.
- [3] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025. URL <https://arxiv.org/abs/2408.08435>.
- [4] LangChain. Deepagents, 2026. URL <https://github.com/langchain-ai/deepagents>. Software framework.
- [5] Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026. URL <https://arxiv.org/abs/2603.28052>.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [7] Yongyuan Liang, Shijie Zhou, Yu Gu, Hao Tan, Gang Wu, Franck Dernoncourt, Jihyung Kil, Ryan A Rossi, and Ruiyi Zhang. Anticipatory planning for multimodal ai agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5925–5935, 2026.
- [8] Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- [9] Jiacheng Liu, Xiaohan Zhao, Xinyi Shang, and Zhiqiang Shen. Dive into claude code: The design space of today’s and future ai agent systems, 2026. URL <https://arxiv.org/abs/2604.14228>.
- [10] Pengfei Liu, Weizhe Yuan, Jinlan Fu, Zhengbao Jiang, Hiroaki Hayashi, and Graham Neubig. Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing, 2021. URL <https://arxiv.org/abs/2107.13586>.

- [11] Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist: Towards fully automated open-ended scientific discovery, 2024. URL <https://arxiv.org/abs/2408.06292>.
- [12] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A survey of context engineering for large language models, 2025. URL <https://arxiv.org/abs/2507.13334>.
- [13] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- [14] MiniMax. Minimax m2.5: Built for real-world productivity, February 2026. URL <https://www.minimax.io/news/minimax-m25>. Official model report.
- [15] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL <https://arxiv.org/abs/2506.13131>.
- [16] OpenAI. Codex, 2026. URL <https://openai.com/codex/>. Product page.
- [17] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- [18] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. Toolllm: Facilitating large language models to master 16000+ real-world apis, 2023. URL <https://arxiv.org/abs/2307.16789>.
- [19] Jiahao Qiu, Xuan Qi, Tongcheng Zhang, Xinzhe Juan, Jiacheng Guo, Yifu Lu, Yimin Wang, Zixin Yao, Qihan Ren, Xun Jiang, Xing Zhou, Dongrui Liu, Ling Yang, Yue Wu, Kaixuan Huang, Shilong Liu, Hongru Wang, and Mengdi Wang. Alita: Generalist agent enabling scalable agentic reasoning with minimal predefinition and maximal self-evolution, 2025. URL <https://arxiv.org/abs/2505.20286>.
- [20] Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>. Official model report and model card for Qwen3.5-35B-A3B.
- [21] Sander Schulhoff, Michael Ilie, Nishant Balepur, Konstantine Kahadze, Amanda Liu, Chenglei Si, Yinheng Li, Aayush Gupta, HyoJung Han, Sevien Schulhoff, Pranav Sandeep Dulepet, Saurav Vidyadhara, Dayeon Ki, Sweta Agrawal, Chau Pham, Gerson Kroiz, Feileen Li, Hudson Tao, Ashay Srivastava, Hevander Da Costa, Saloni Gupta, Megan L. Rogers, Inna Goncearenco, Giuseppe Sarli, Igor Galynker, Denis Peskoff, Marine Carpuat, Jules White, Shyamal Anadkat, Alexander Hoyle, and Philip Resnik. The prompt report: A systematic survey of prompt engineering techniques, 2025. URL <https://arxiv.org/abs/2406.06608>.

- [22] Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models’ sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting, 2024. URL <https://arxiv.org/abs/2310.11324>.
- [23] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [24] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. In *International Conference on Learning Representations*, volume 2025, pages 65882–65919, 2025.
- [25] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. URL <https://arxiv.org/abs/2201.11903>.
- [26] Siyuan Xu, Shiyang Li, Xin Liu, Tianyi Liu, Yixiao Li, Zhan Shi, Zixuan Zhang, Zilong Wang, Qingyu Yin, Jianshu Chen, et al. Controllable and verifiable tool-use data synthesis for agentic reinforcement learning. *arXiv preprint arXiv:2604.09813*, 2026.
- [27] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search, 2025. URL <https://arxiv.org/abs/2504.08066>.
- [28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [30] Xunjian Yin, Xinyi Wang, Liangming Pan, Li Lin, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursively self-improvement. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 27890–27913, 2025. doi: 10.18653/v1/2025.acl-long.1354. URL <https://aclanthology.org/2025.acl-long.1354/>.
- [31] Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation, 2024. URL <https://arxiv.org/abs/2310.02304>.
- [32] Hangfan Zhang, Siyuan Xu, Zhimeng Guo, Huaisheng Zhu, Shicheng Liu, Xinrun Wang, Qiaosheng Zhang, Yang Chen, Peng Ye, Lei Bai, et al. The path of self-evolving large language models: Achieving data-efficient learning via intrinsic feedback. *arXiv preprint arXiv:2510.02752*, 2025.
- [33] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents, 2025. URL <https://arxiv.org/abs/2505.22954>.
- [34] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. Agentic context engineering: Evolving contexts for self-improving language models, 2026. URL <https://arxiv.org/abs/2510.04618>. ICLR 2026.
- [35] Shao Zhang, Xihuai Wang, Wenhao Zhang, Chaoran Li, Junru Song, Tingyu Li, Lin Qiu, Xuezhi Cao, Xunliang Cai, Wen Yao, Weinan Zhang, Xinbing Wang, and Ying Wen. Leveraging dual process theory in language agent framework for real-time simultaneous human-AI collaboration. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*

- (*Volume 1: Long Papers*), pages 4081–4108, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.206. URL <https://aclanthology.org/2025.acl-long.206/>.
- [36] Ningyan Zhu, Huacan Wang, Jie Zhou, Feiyu Chen, Shuo Zhang, Ge Chen, Chen Liu, Jiarou Wu, Wangyi Chen, Xiaofeng Mou, and Yi Xu. Semaclaw: A step towards general-purpose personal ai agents through harness engineering, 2026. URL <https://arxiv.org/abs/2604.11548>.
- [37] Mingchen Zhuge, Wenyi Wang, Louis Kirsch, Francesco Faccio, Dmitrii Khizbullin, and Juergen Schmidhuber. Language agents as optimizable graphs, 2024. URL <https://arxiv.org/abs/2402.16823>.

A Additional Implementation Details

A.1 Experimental details

Model inference services MiniMax M2.5 and GLM-5 were accessed through hosted inference services, using MiniMax’s hosted API and OpenRouter, respectively.² Qwen3.5-35B-A3B was deployed locally on four NVIDIA H200 GPUs using an internal image derived from the public SGLang Docker image lmsysorg/sglang:v0.5.12-cu129.³

Terminal-Bench-2.0 configuration. We use Harbor⁴ as the execution environment for all Terminal-Bench-2.0 tasks. Evaluations are run on an isolated machine with 64 CPU cores, 256 GB of memory, and a 2 MB/s outbound network bandwidth cap. We use a default concurrency of 32 tasks for MiniMax M2.5 and GLM-5, and a concurrency of 48 tasks for the locally deployed Qwen3.5-35B-A3B backend. To reduce failures caused by incidental network latency rather than agent behavior, we mirror a subset of external resources required by Terminal-Bench-2.0 tasks when the original resources are stable but slow to download. Tasks that depend on external resources that cannot be accessed reliably, as well as tasks requiring multimodal inputs unsupported by the initial harness, are excluded from the main 64-case evaluation set. This configuration keeps the benchmark environment controlled while preserving the core terminal-interaction setting of Terminal-Bench-2.0.

A.2 Additional GLM-5 analysis

Failed trace under the initial harness	Successful trace under the edited harness
<pre>T072 Todo: Find and download POV-Ray 2.2 source archives; extract source; build/install; test with a sanity command. T151 Initial povray.org download attempt prints: Failed from povray.org. T190 /tmp/povray-2.2.tar.gz is only 263 bytes. T244 gzip: stdin: not in gzip format; tar: Error is not recoverable. T265 Downloaded content is an HTML 404 Not Found page from povray.org. T322 curl web.archive.org ... povray-2.2.tar.gz ERROR: Command timed out after 300 seconds. T388 404: Not Found T507 /tmp/povray-2.2.tar.gz is 278698 bytes, but it is not the archive. T533 gzip: stdin: not in gzip format; tar extraction fails again. T580 File preview starts with <doctype html> rather than archive bytes. T1250 Another mirror lists /repo/pkgsrc/povray/povray-3.7.0.8.tar.gz. T1641 Mirror provides POV-Ray 3.7 archives, not the required 2.2 release. T2054 curl codeload.github.com/POV-Ray/povray ... ERROR: Command timed out after 300 seconds. # long monolithic external operations timed out T2303 Eventually finds Index of /packages/povray/Old-Versions/Official-2.2. T2360 Downloads POVSRG.TAR.Z and POVDOC.TAR.Z with expected sizes. T2480 Extracts source files, docs, and demos from the archives. T2677 UNIX docs warn POV-Ray may not compile directly as-is. T2701 Reads UNIX makefile for Persistence of Vision Raytracer. T2928 Installs build dependencies, edits Makefile/config.h, and compiles the old C source with warnings. T3331 Installs binary to /usr/local/bin/povray. T3383 Todo: Test build with sanity check command -> in progress. T3411 Render stats printed, but command failed with exit code 150. T3458 Render stats printed, but command failed with exit code 150. T3506 Render stats printed, but command failed with exit code 166. T3546 A /tmp/test.tga output file exists despite non-zero runs. T3602 Render stats printed again; Exit code: 180. S3906 Finalized anyway: non-zero exit codes are due to old signal handling; the actual rendering works correctly. # finalized despite non-zero sanity checks</pre>	<pre>5003 I'll build POV-Ray 2.2 from source. Let me start by exploring the workspace and finding the source archives. T013 Workspace contains only deps; source archives are not present locally. 5022 The source archives aren't present locally. Let me download POV-Ray 2.2. T030 povuni_s.tgz download is a 404 Not Found HTML page. T035 curl web.archive.org ... povuni_s.tgz ERROR: Command timed out after 120 seconds. 5037 Let me try a different approach - searching for alternative mirrors: # staged work and bounded timeout 5599 I found the ICM FTP server has an Old-Versions directory. Let me explore it: 5604 Excellent! Found the Official-2.2 directory. Let me check its contents: 5658 The source archive POVSRG.TAR.Z exists. Let me verify it and find the documentation files: T741 Index of /ftp/pub/povray/Old-Versions/Official-2.2. T746 HEAD check: POVSRG Content-Length 575233; HTTP 200 OK. T747 HEAD check: POVDOC Content-Length 276500; HTTP 200 OK. T753 Downloaded /tmp/POVDOC_test.tgz, 276500 bytes. T755 Downloaded /tmp/POVSRG_test.tgz, 575233 bytes. # checked early output before committing more work 5762 Found working URLs. Let me download and extract the archives: T777 Downloads archives, checks magic bytes, and extracts source/docs. T915 Installs missing build tools; compilation proceeds with C warnings. T950 Installed binary to /usr/local/bin/povray. T955 Initial include directory check fails: /app/povray-2.2/povdoc/include. T960 Finds actual include path: /app/povdoc/include. T970 Include directory contains chars.inc, include.cat, rough.gif, test.gif. T980 Cannot open include file: shapes.inc Command failed with exit code 1. T985 Sample scene includes shapes.inc and colors.inc. T1015 Rechecks extracted doc/demo tree with include files available. T1025 Resolution 100 x 100 Command succeeded with exit code 0. # repaired failed sanity check before finalizing T1030 Todo: download source archive; extract; configure/compile; install; sanity test -> all completed. 51032 POV-Ray 2.2 has been successfully built and installed. The sanity check rendered the test scene.</pre>

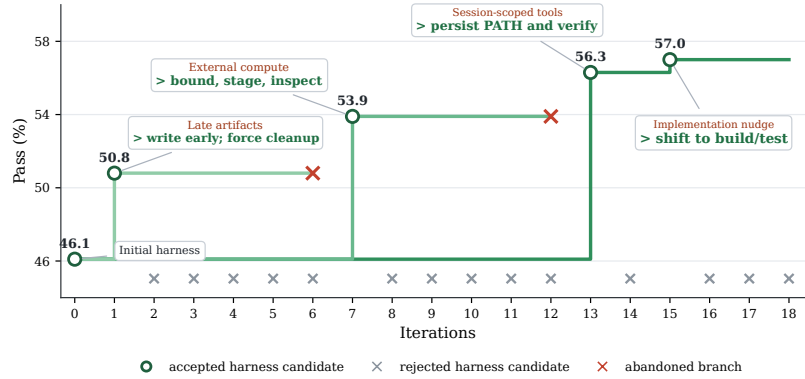
S = agent step T = tool call/result

Figure 9: Case study of a GLM-5 harness edit on the Terminal-Bench-2.0 build-pov-ray task. Left: a failed trace under the initial harness, where long monolithic external downloads consume large tool budgets and the agent later finalizes despite repeated non-zero sanity checks. Right: a successful trace under the edited harness, where the agent uses bounded staged operations, checks external archive evidence before committing more work, and repairs the failed sanity check before finalizing.

²MiniMax token plan: <https://platform.minimax.io/subscribe/token-plan>; OpenRouter GLM-5 endpoint: <https://openrouter.ai/z-ai/glm-5>. Accessed on 2026/05.

³<https://hub.docker.com/r/lmsysorg/sglang/tags?name=v0.5.12-cu129>.

⁴<https://www.harborframework.com/docs/tutorials/running-terminal-bench>.



(a) Self-Harness evolution trajectory. Green markers denote accepted harness candidates and gray crosses denote rejected harness candidates. Step lines connect accepted candidates and keep performance flat across rejected iterations. Two early branches were discarded due to no further improvement. The remaining lane becomes the final accepted harness.

```

GLM-5: persistent path export
1 1 def build_bootstrap_instruction() -> str:
2   - return "Start by inspecting the workspace and identifying the smallest relevant edit surface."
3   + return (
4     + "Start by inspecting the workspace and identifying the smallest relevant edit surface. "
5     + "When installing tools or modifying environment variables, ensure changes persist across shell sessions: "
6     + "add executables to system paths such as /usr/local/bin, use /etc/profile.d scripts or ~/.bashrc "
7     + "for PATH modifications instead of session-scoped exports, and verify tool accessibility with "
8     + "'which <tool>' or by executing the tool directly."
9   )

GLM-5: transition from exploration to implementation
1 1 def build_verification_instruction() -> str:
2   - return "Before concluding, verify the result with the most targeted command, file read, or test you can run."
3   + return (
4     + "Before concluding, verify the result with the most targeted command, file read, or test you can run. "
5     + "If you have been exploring without producing the required artifacts, transition to implementing "
6     + "and testing a solution."
7   )

```

(b) Differences for the four harness modifications accepted by Self-Harness and retained in the final harness. Red rows denote code removed from the initial harness, and green rows denote the updated harness behavior.

Figure 10: GLM-5 Self-Harness run. Panel (a) summarizes the Self-Harness evolution trajectory, while panel (b) expands the accepted updates from this trajectory into their retained code-level edits in the final harness.